

Exerciții — tabel cu coloane făcute de tine

Clasă, listă, DataGridView — pas cu pas.

Toată fișa se face pe **o singură formă**, `MasiniForm`, și pe **o singură clasă**, `Masina`. Fiecare exercițiu adaugă un buton nou lângă celelalte. Nu porni proiect nou și nu șterge ce ai făcut la exercițiul anterior.

Ca să pornești forma, în `Program.cs` linia trebuie să fie `Application.Run(new MasiniForm());`.

Tiparul de la care pleci

Ăsta e modelul. Toate exercițiile sunt variații pe el — citește-l o dată întreg.

Clasa, în fișierul ei, `Masina.cs` :

```
class Masina
{
    public string marca;
    public string model;
    public int an;
    public double pret;
    public bool disponibila;
}
```

Coloanele, scrise de tine:

```
private void CreateColumns()
{
    dgvMasini.Columns.Clear();
    dgvMasini.Columns.Add("marca", "Marca");
    dgvMasini.Columns.Add("model", "Model");
    dgvMasini.Columns.Add("an", "An");
    dgvMasini.Columns.Add("pret", "Pret");
    dgvMasini.Columns.Add("disponibila", "Disponibila");
}
```

`Columns.Add(nume, antet)` primește **două** lucruri: numele tehnic, cu care găsești coloana din cod, și antetul, care se vede pe ecran. Pot fi diferite.

`CreateColumns()` se cheamă **o singură dată**, din constructorul formei, imediat după `InitializeComponent()`.

Umplerea, la apăsarea butonului:

```

dgvMasini.Rows.Clear();
for (int i = 0; i < masini.Count; i++)
{
    Masina m = masini[i];
    dgvMasini.Rows.Add(m.marca, m.model, m.an, m.pret, m.disponibila);
}

```

 **Rows.Add** pune valorile **în ordinea coloanelor**, nu după nume. Dacă schimbi ordinea în **CreateColumns** și uiți să schimbi și aici, datele intră pe coloana greșită — programul compilează, rulează și minte.

Așa arată rezultatul, pe alt set de date:

Incarca userii		Incarcati 10 useri			
Username	Email	Password	Telefon	Varsta	Valid
andrei_pop	andrei.popescu@gm...	Andrei99!	0722123456	25	True
elena_io	elena.ionescu@yah...	ElenaSecure20	0731987654	31	True
vlad_m	vlad.marcu@outlook...	vladmarcu123	0745112233	19	False
ralu_c	raluca.constantin@g...	Ralu*2026	0756445566	42	True
cosmin_radu	cosmin.radu@gmail....	cosminPass!	0767889900	28	True
ana_stan	anamaria.stan@yah...	ana_maria_99	0721334455	22	True
marian_d	marian.dumitru@gma...	marianD76	0732556677	50	False
AlexGamer	alex.neagu@outlook...	AlexGaming!!	0743778899	16	True
cristina_v	cristina.vlad@gmail....	CrisVlad85	0754114477	35	True
stefan_b	stefan.badea@yaho...	stefanB_99	0765225588	27	True

Același tipar, aplicat pe useri — asta trebuie să-ți iasă

Partea 1 — tabelul care se umple

1. Prima încărcare

1. Adaugă în proiect fișierul `Masina.cs` și scrie clasa de mai sus.
2. Pe formă pune un `DataGridView` numit `dgvMasini` și un buton numit `btnIncarca`. Numele se pun **înainte** de dublu-click.
3. Pe `dgvMasini` setează din Properties: `ReadOnly = True`, `AllowUserToAddRows = False`, `RowHeadersVisible = False`, `AutoSizeColumnsMode = Fill`.
4. Scrie metoda `CreateColumns()` și cheam-o din constructor.
5. Scrie o metodă `IncarcaMasini()` care întoarce un `List<Masina>` cu **opt** mașini, de mărci diferite, cu ani și prețuri diferite, unele indisponibile.
6. În `btnIncarca_Click`, cheamă `IncarcaMasini()` și umple tabelul cu bucla din tipar.

Gata când: apeși butonul **de trei ori la rând** și în tabel sunt tot opt rânduri.

2. Câte sunt

Pune o etichetă `lblStare` lângă buton. După încărcare, scrie în ea câte mașini sunt în tabel. la numărul din listă, nu scrie `8` de mâna.

3. Antete scrise omenește

Schimbă antetele în `Pretul (lei)`, `Anul fabricatiei`, `In stoc`. Numele tehnice ale coloanelor rămân `pret`, `an`, `disponibila`.

Gata când: antetele arată textele noi, iar în cod ai în continuare `dgvMasini.Columns["pret"]` și funcționează.

4. O coloană care nu există în clasă

Adaugă în `CreateColumns` o a șasea coloană, cu numele `vechime` și antetul `Vechime (ani)`. Clasa `Masina` **nu** primește niciun câmp nou.

La umplere, calculează valoarea: anul curent este 2026.

Gata când: o mașină din 2018 arată `8` pe coloana de vechime.

5. Butonul apăsat de multe ori

Scoate `dgvMasini.Rows.Clear();` din `btnIncarca_Click` și apasă butonul de trei ori. Scrie într-o etichetă câte rânduri sunt acum. Apoi pune linia la loc.

Partea 2 — prelucrări înainte de afișare

De aici încolo, fiecare exercițiu are butonul lui. Tabelul și coloanele rămân aceleași.

6. Doar cele disponibile

Buton `btnDisponibile` : afișează în tabel doar mașinile cu `disponibila` pe `true` .

Golești tabelul, parcurgi lista cu `for` , și adaugi rândul **doar dacă** mașina trece condiția.

7. Sub un preț dat

Buton `btnSubPret` și o casetă `txtPret` . Afișează doar mașinile mai ieftine decât suma tastată.

Dacă în casetă nu e un număr, apare un `MessageBox` și tabelul rămâne neatins.

Gata când: scrii `abc` , apeși, și aplicația rămâne deschisă cu tabelul nemodificat.

8. Ordonate după preț

Buton `btnOrdonate` : afișează mașinile de la cea mai ieftină la cea mai scumpă.

Ordonarea o faci tu, cu două bucle `for` și o variabilă de schimb, ca la C++. Fără `Sort` , fără `OrderBy` .

9. Prețul mediu

Buton `btnMedia` : calculează prețul mediu al celor opt mașini și scrie-l într-o etichetă. Tabelul nu se schimbă.

10. Cea mai scumpă, cea mai veche

Buton `btnExtreme` : într-o etichetă scrie marca și modelul celei mai scumpe mașini, iar în alta pe cea mai veche. Dacă lista ar fi goală, programul nu are voie să crape.

Partea 3 — tabelul care răspunde

11. Caută după marcă

Buton `btnCauta` și o casetă `txtCauta`. Afișează doar mașinile a căror marcă e cea tastată.

Dacă nu găsește niciuna, tabelul rămâne gol și o etichetă anunță asta. Nu un tabel gol fără explicație.

12. Rândul selectat

Sub tabel pune trei etichete. Când selectezi un rând, ele arată marca, modelul și prețul mașinii alese.

1. Ține lista într-un câmp al formeii, nu într-o variabilă locală — altfel după `btnIncarca` n-o mai ai.
2. Folosește evenimentul `SelectionChanged` al tabelului.
3. Verifică `CurrentRow` înainte să-l folosești.

Gata când: te muți prin tabel cu **săgețile de pe tastatură** și etichetele se schimbă și așa, nu doar la click.

13. Ștergerea

Buton `btnSterge`: scoate din listă mașina selectată și reafixează tabelul.

Gata când: apeși `Sterge` **înainte** să fi încărcat ceva și aplicația rămâne deschisă.

14. Adăugarea

Un `GroupBox` cu cinci controale: patru casete pentru marcă, model, an, preț și o bifă pentru disponibilitate. Buton `btnAdauga`.

Reguli, toate verificate înainte de a adăuga ceva:

- marca și modelul nu pot fi goale;
- anul e un număr întreg între 1950 și 2026;
- prețul e un număr pozitiv.

Dacă una nu e respectată, apare un mesaj care spune **care** regulă a fost încălcată, iar tabelul rămâne neatins.

Gata când: adaugi o mașină corectă și apare în tabel; încerci cu anul **3000** și numărul de rânduri nu se schimbă.

Înainte să-mi zici că e gata

Treci fiecare buton prin cele cinci:

- apeși butonul **de trei ori la rând** — a treia apăsare face același lucru ca prima?
- lași casetele goale și apeși — apare un mesaj, sau se închide aplicația?
- apeși butoanele care lucrează cu selecția **înainte** să încarci ceva?
- ai vreun handler numit `button1_Click` ?
- `CreateColumns()` se cheamă o singură dată, din constructor — nu la fiecare apăsare?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro